

SANTA CLARA UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Date: May 1, 2026

I HEREBY RECOMMEND THAT THE THESIS PREPARED UNDER MY SUPERVISION BY

Kenneth Kang

ENTITLED

ML Models for Speech Diarization

BE ACCEPTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF

BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

Thesis Advisor [Sean Choi]

Department Chair [Silvia Figueira]

ML Models for Speech Diarization

by

Kenneth Kang

Submitted in partial fulfillment of the requirements
for the degree of
Bachelor of Science in Computer Science and Engineering
School of Engineering
Santa Clara University

Santa Clara, California
May 1, 2026

ML Models for Speech Diarization

Kenneth Kang

Department of Computer Science and Engineering
Santa Clara University
May 1, 2026

ABSTRACT

A good abstract is a concise summary (1–2 paragraphs) of the entire project: introduction, problem statement, work accomplished, results, conclusions, and recommendations. When you write the abstract, imagine that the reader will not read anything else but that you must get your major point across immediately. This requires the efficiency of words and phrases. An abstract is written to stand alone, without jargon or references to figures and tables in the report body.

Don't forget to delete this explanatory red text/block introduction, and replace it with your text in black.

Table of Contents

1	Introduction	1
1.1	Problem Statement	2
1.2	Objectives	2
1.3	Our approach	3
2	User Research - example middle chapter	5
2.1	Methods	5
2.2	Stakeholder needs	6
2.3	User stories	6
3	Design and Rationale - example middle chapter	8
3.1	Design	8
3.2	Use Case Overview	10
3.3	Use Case Descriptions	10
3.4	Use Case Diagram(s)	15
3.5	Class Diagram	16
3.6	Sequence Diagram	17
3.7	Activity Diagram	18
3.8	Functional requirements	19
3.9	Non-functional requirements	19
3.10	Rationale	19
4	Technologies - example middle chapter	21
4.1	System Components	21
5	System Evaluation - example middle chapter	23
5.1	Internal Testing	24
5.2	External Testing	25
5.3	Unit Testing	26
5.4	Integration Test(s)	26
6	Implementation Plan - example middle chapter	28
6.1	Timeline	28
6.2	Project Risks	29
7	Constraints and Standards	31
7.1	Constraints	31
7.2	Standards	33

8	Societal Issues	35
8.1	Ethical	35
8.2	Social	35
8.3	Economic	36
8.4	Health and Safety	36
8.5	Usability	36
8.6	Compassion	37
9	Conclusion	38
10	Acknowledgments	39
11	References	40
12	Appendices	41

List of Figures

3.1	Use Case 1: Run Real-Time Courtroom Diarization	10
3.2	Use Case 2: View and Correct Speaker Timeline	11
3.3	Use Case 3: Export Annotated Courtroom Transcript	12
3.4	Use Case 4: Evaluate Diarization Model Performance	13
3.5	Use Case 5: Retrain Model with Corrected Courtroom Data	14
3.6	UML Use Case Diagram: ML Speech Diarization	15
3.7	Class Diagram	16
3.8	Sequence Diagram	17
3.9	Activity Diagram	18
6.1	ML Speech Diarization Timeline Diagram	28

Overall, have at least 20 figures at the end of the senior design project.

Don't forget to delete this explanatory red text/block introduction, and replace it with your text in black.

Chapter 1

Introduction

In modern judicial systems, accurate and timely transcription of courtroom proceedings is essential for ensuring transparency, fairness, and accountability. However, the process of transcribing spoken dialogue in real time presents significant challenges, especially in Mandarin, where tonal variations and homophones can lead to transcription ambiguity. Current systems, such as the Yawei stenography system, have been used in Chinese courts for a long time to facilitate typing and transcription. Yet, despite their efficiency, these systems rely heavily on manual input and the user's ability to select the correct Chinese character from multiple homophonous options, which can result in inconsistencies, slower transcription, and potential misinterpretations in legal records.

As courtrooms evolve toward automation and digital recordkeeping, there is a growing demand for machine learning (ML) based speech diarization, which automatically determines “who spoke when” to enhance both the accuracy and efficiency of courtroom transcripts. In Mandarin, however, speaker diarization remains underexplored. Existing diarization models, typically trained on English datasets, often struggle with multi-speaker overlap, tonal differentiation, background noise, and the dynamic nature of courtroom dialogue. These limitations make it difficult to apply generic speech recognition tools effectively in Mandarin judicial contexts.

Our project, “Machine Learning Models for Speech Diarization,” aims to bridge this technological gap by developing and refining diarization models focusing more on Mandarin speech patterns and the unique characteristics of courtrooms. The ultimate goal is to reduce reliance on the Yawei input system and minimize the ambiguity caused by manual character selection, thereby improving transcription precision and reducing the workload on court reporters. Which, in turn, can translate into usage outside the courthouse and enhance user experiences when tuning in to live video formats, or even isolate speakers for those with hearing disabilities.

1.1 Problem Statement

Accurate courtroom transcriptions currently depend on a task known as speaker diarization. In a Mandarin-language courtroom, this task is particularly challenging: multiple people, including judges and attorneys, often speak in rapid sequence, sometimes overlapping. It is estimated that at least 95% of Mandarin characters have a homophone. Additionally, due to the many different dialects of Mandarin spoken around the globe, Mandarin can sound quite different from person to person. Because of this, traditional Mandarin stenographic systems such as Yawei rely heavily on manual intervention for selection of the characters that have homophones. This is extremely time-consuming and introduces a lot of delay into the transcription workflow.

Our senior design project seeks to address these shortcomings by developing a specialized machine learning pipeline tuned for Mandarin diarization within courtroom contexts. The goal is to significantly reduce the amount of intervention compared to current off-the-shelf systems. We will gather Mandarin multi-speaker audio, preferably in courtroom-like or meeting-style settings, and after diarization, our model will be trained based on this information.

Success will be judged on our intervention reduction percentage as well as the general accuracy of the model in listening to different speakers. Comparisons will be made between currently existing models/approaches, as well as human stenographers.

1.2 Objectives

The objective of this project is to design, implement, and evaluate a machine learning pipeline for speech diarization in child-adult audio. Speech diarization focuses on identifying “who spoke when,” but our project applies this problem to a more specific and challenging setting: conversations between children and adults. Parent-child and child-directed speech often includes overlapping voices, interruptions, background noise, short speech turns, and high-pitched or inconsistent child vocalizations. These characteristics make the task more difficult than standard adult speaker diarization, so our project aims to study how existing diarization tools perform in this setting and how their outputs can be improved.

One major objective is to collect and prepare audio data that represents realistic child-adult speech conditions. This includes gathering usable audio clips, extracting audio from public video sources when needed, organizing files in a consistent structure, and preparing the data for experimentation. Because raw audio can vary in quality, length, format, and clarity, the project must also include preprocessing steps that make the audio more usable for diarization.

These steps may include file conversion, segmentation, cleaning, and maintaining timestamp alignment so that the results can be reviewed and evaluated consistently.

Another objective is to establish a baseline for comparison. Before making improvements, the project must first test existing diarization models on the selected audio clips. This allows us to identify common failure cases, such as confusing child speech with adult speech, missing overlapping speech, or creating speaker labels that are inconsistent across a clip. By comparing model output to manually reviewed or labeled segments, we can better understand where the system succeeds and where it needs improvement.

The final objective is to evaluate whether the improved pipeline produces better results than the baseline system. The project will use diarization-focused evaluation metrics such as Diarization Error Rate, while also considering qualitative issues such as overlap handling and label clarity. Since this project is a senior design research prototype, the goal is not to create a finished commercial product. Instead, the goal is to build a working and reproducible system that demonstrates the challenges of child-adult diarization, measures improvement where possible, and clearly documents the limitations that future work should address.

1.3 Our approach

Our approach is to build the project as an end-to-end research pipeline for child-adult speech diarization. Instead of focusing only on a single model or a visual application, the system connects each major stage of the workflow, including audio collection, preprocessing, diarization, evaluation, and reporting. This approach is appropriate because the quality of diarization results depends not only on the model itself, but also on how the audio is collected, cleaned, organized, labeled, and evaluated.

The project begins by collecting audio clips that include adult speech, child speech, child-directed speech, and overlapping speech. These clips are then processed into a consistent format so that they can be used across multiple experiments. Once the audio is prepared, the system runs baseline diarization tools on the same set of clips. This baseline stage is important because it gives the project a measurable starting point and helps identify where existing systems struggle. In child-adult audio, common problems include missed speaker changes, incorrect labels, noisy segments, and difficulty separating overlapping voices.

After the baseline is established, the project focuses on improving the diarization workflow through better data preparation, clearer labeling, repeated testing, and performance comparison. When model outputs are incorrect, the team can review the segments, document failure cases, and use that information to guide later improvements. This makes the project iterative rather than one-time. Each experiment helps the team better understand how the system behaves and what changes may improve the results.

This approach is different from simply applying an existing diarization model because it focuses on a specific and underexplored use case. Child-adult speech has unique challenges that are not always well represented in general diarization datasets. By building a modular pipeline, the project can isolate problems at different stages and evaluate improvements in a controlled way. This makes the system practical for a senior design timeline while still allowing the team to produce meaningful results and recommendations for future development.

Chapter 2

User Research - example middle chapter

2.1 Methods

The methods used in this project are based on technical research, audio preparation, manual review, model testing, and quantitative evaluation. Since the project focuses on child-adult speech diarization, the first step was to understand the characteristics that make this type of audio difficult to process. Child-adult conversations often include interruptions, overlapping speech, background noise, short responses, and variation in children's voices. These issues shaped the way we selected audio clips, designed the pipeline, and evaluated model performance.

A major method used in the project is audio collection and preparation. The system uses audio clips that can be extracted, organized, and passed through the diarization pipeline. In some cases, this involves converting public video sources into audio files and numbering them consistently so that each file can be traced throughout the experiment. This organization is important because the same clips must be used for baseline testing, improved model testing, and final comparison. Without a consistent file structure, it would be difficult to reproduce results or understand which outputs belong to which original audio source.

Manual review is another important method because diarization quality cannot be judged only by whether the code runs successfully. The team must listen to selected clips and compare the system's output against what is actually happening in the audio. This process helps determine whether the model correctly separates adult speech, child speech, and overlapping speech. It also helps reveal cases where the timestamps are misaligned, speaker labels are inconsistent, or the model fails because of noise or unclear speech.

The project also uses quantitative evaluation to measure performance. The main metric is Diarization Error Rate, which provides a way to compare baseline outputs with improved outputs. However, because this project focuses on child-adult speech, evaluation also considers overlap handling and the quality of labels for difficult segments. Combining manual review with quantitative metrics allows the project to evaluate the system in a more complete way.

The goal is not only to produce diarization output, but to determine whether that output is accurate, understandable, and useful for future analysis.

2.2 Stakeholder needs

The main stakeholders for this project are the researchers, engineers, and future users who need to analyze child-adult speech more efficiently. The first stakeholder group is the machine learning team developing the system. For this group, the system needs to be reproducible, modular, and measurable. The team must be able to collect audio, preprocess files, run diarization models, save outputs, compare results, and document whether changes improve performance. A clear pipeline is important because it allows the team to identify where problems occur and prevents the evaluation process from becoming inconsistent.

Another stakeholder group is speech, language, or child-development researchers who may want to study interactions between children and adults. These users need timestamped speaker labels that help them understand who is speaking and when. Their main need is not only accuracy, but also interpretability. If the system produces speaker labels that are unclear or inconsistent, then the output may not be useful for research. These users would benefit from a system that reduces manual listening time while still making difficult or uncertain segments easy to identify.

Technical reviewers and project evaluators are also stakeholders. They need to understand whether the system is reliable, whether the evaluation is fair, and whether the conclusions are supported by evidence. For these stakeholders, the project must include clear metrics, documented methods, and examples of both successful and unsuccessful outputs. This is especially important because a diarization system can appear useful even when it makes serious mistakes. Transparent evaluation helps prevent the project from overstating its results.

Children and families are indirect stakeholders because their voices may be represented in the audio data. Even when audio comes from public or approved sources, the project still has a responsibility to handle the data carefully. This means limiting access to audio files, avoiding unnecessary personal information, and reporting results in a way that does not expose individuals. Because of this, the project prioritizes responsible data handling along with technical performance. The system is designed primarily for researchers and engineers, but it must still respect the privacy and dignity of the people represented in the data.

2.3 User stories

Court Reporter

- **As a court reporter**, I want the system to display a real-time speaker timeline with labeled segments (e.g., Judge, Attorney A, Witness), **so that** I can follow the dialogue and only correct ambiguous segments instead of

typing everything manually.

- **As a court reporter**, I want to quickly click on mis-labeled segments and reassign them to the correct speaker, **so that** I can reduce diarization errors without disrupting the flow of the hearing.

Court Clerk or Judge

- **As a court clerk or judge**, I want to export an annotated transcript with speaker labels and timestamps, **so that** I can reliably review who said what during the hearing without replaying the entire audio.

ML engineer

- **As an ML engineer**, I want to run evaluation jobs that compare our model's diarization accuracy and required interventions against existing systems and human stenographers, **so that** we can quantitatively measure improvement over baseline approaches.
- **As an ML engineer**, I want to feed corrected courtroom recordings back into the training pipeline, **so that** the diarization model gradually adapts to Mandarin accents, dialects, and courtroom-specific speaking patterns.

Chapter 3

Design and Rationale - example middle chapter

3.1 Design

The system is designed as a modular speech diarization pipeline. At a high level, the pipeline includes audio collection, preprocessing, diarization, evaluation, and reporting. This structure supports the goals of the project because child-adult diarization is not only a model performance problem. It also depends on the quality of the input audio, the consistency of preprocessing, the clarity of labels, and the reliability of the evaluation process. By separating the system into modules, each stage can be tested, revised, and improved without requiring the entire project to be rebuilt.

The first part of the design is the audio input component. This component is responsible for collecting and organizing audio clips that include adult speech, child speech, child-directed speech, and overlapping speech. The system must keep these files organized so that each input can be connected to its output during testing. Clear file naming and numbering are important because they allow the team to reproduce experiments and compare different model runs on the same set of clips.

The second part of the design is the preprocessing component. This stage prepares the audio before it is passed into the diarization model. Preprocessing may include converting file formats, trimming audio, checking sample rates, segmenting longer clips, and preserving timestamp alignment. This step is necessary because poor input quality can negatively affect diarization results. Separating preprocessing from model inference also makes it easier to identify whether errors are caused by the audio data, the preprocessing workflow, or the model itself.

The third part of the design is the diarization and evaluation component. The diarization model generates timestamped speaker segments, while the evaluation component compares those results against manually reviewed labels or selected ground-truth segments. The system is designed to support both baseline testing and later improved testing, allowing the team to measure whether changes to the pipeline actually improve performance. The final reporting com-

ponent stores outputs, metrics, logs, and examples of failure cases. Together, these components create a repeatable workflow that supports research, testing, and future improvement.

3.2 Use Case Overview

The use cases were derived by taking each high-level user story and expanding it into a concrete interaction between an actor and the Mandarin Courtroom Speech Diarization System. The court reporter’s stories about needing real-time speaker labels and easy correction became the “Run Real-Time Courtroom Diarization” and “View and Correct Speaker Timeline” use cases. The clerk/judge story about needing reviewable records became “Export Annotated Courtroom Transcript,” and the ML team’s stories about measuring improvement and continuously refining the model became “Evaluate Diarization Model Performance” and “Retrain Model with Corrected Courtroom Data.”

3.3 Use Case Descriptions

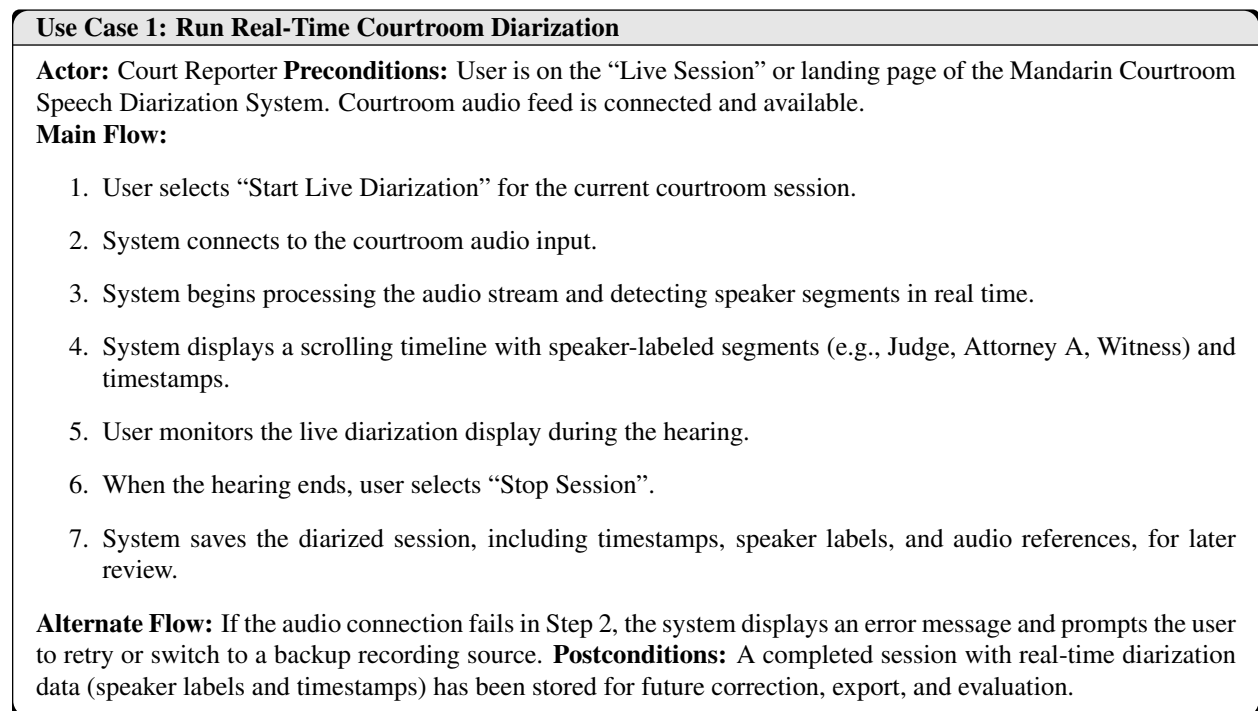


Figure 3.1: Use Case 1: Run Real-Time Courtroom Diarization

Use Case 2: View and Correct Speaker Timeline

Actor: Court Reporter **Preconditions:** A courtroom session has already been diarized and stored by the system.

Main Flow:

1. User navigates to the “Session History” page and selects a past session.
2. System displays the audio waveform and aligned speaker segments on a timeline.
3. User clicks on a segment that appears to have an incorrect speaker label.
4. System opens a dialog showing the current speaker label and a list of available speakers (e.g., Judge, Clerk, Attorney A/B, Witness).
5. User selects the correct speaker from the list and confirms the change.
6. System updates the speaker label on the timeline, and visually refreshes the segment.
7. User repeats Steps 3–6 for any additional segments that require correction.
8. User selects “Save Corrections”.
9. System stores the corrected labels and logs which segments required manual intervention.

Alternate Flow: If a segment is too noisy or overlapping and the user cannot determine the correct speaker, the user marks the segment as “Uncertain”, and the system records this status for future research analysis. **Postconditions:** The diarized session now has corrected speaker labels, and intervention data is recorded for performance evaluation and potential model retraining.

Figure 3.2: Use Case 2: View and Correct Speaker Timeline

Use Case 3: Export Annotated Courtroom Transcript

Actor: Court Clerk / Judge **Preconditions:** A diarized session with speaker labels exists. If automatic speech recognition (ASR) is integrated, text for each segment has already been generated or is available.

Main Flow:

1. User navigates to “Session History” and selects a completed session.
2. User selects the “Export Transcript” option.
3. System prompts the user to choose an export format (e.g., PDF, Word, plain text, or JSON).
4. System composes an ordered transcript in the chosen format, including timestamps, speaker labels, and (if available) recognized text for each segment.
5. System generates the transcript file.
6. System displays a confirmation message and provides a link or button to download the file.
7. User downloads the file and saves it to the court document repository.

Alternate Flow: If no text transcript is available (diarization only), the system informs the user and offers an export of timestamps and speaker labels only (e.g., CSV/JSON with [start time, end time, speaker]). **Postconditions:** A transcript or speaker-label file has been generated and downloaded, ready for archival, review, or legal reference.

Figure 3.3: Use Case 3: Export Annotated Courtroom Transcript

Use Case 4: Evaluate Diarization Model Performance

Actor: ML Engineer **Preconditions:** The system has stored diarized sessions, ground-truth labels (from human annotators or stenographers) for at least some sessions, and logs of manual corrections.

Main Flow:

1. User navigates to the “Evaluation” or “Analytics” section.
2. System displays available datasets/sessions and evaluation options.
3. User selects one or more sessions and chooses a baseline (e.g., “Existing Yawei workflow” or “Baseline diarization model”).
4. User selects metrics such as Diarization Error Rate (DER), speaker confusion rate, and manual intervention rate.
5. System runs the evaluation, comparing model output against ground-truth labels and intervention logs.
6. System displays evaluation results in tables and/or charts (e.g., DER vs. baseline, interventions per hour).
7. User may export the evaluation report for documentation (e.g., CSV or PDF).

Alternate Flow: If some selected sessions lack complete ground-truth labels, the system warns the user and allows them to (a) exclude those sessions or (b) proceed with a partial evaluation on the labeled portions only.

Postconditions: Evaluation metrics are computed and stored, providing quantitative evidence of accuracy and intervention reduction compared with existing systems and human stenographers.

Figure 3.4: Use Case 4: Evaluate Diarization Model Performance

Use Case 5: Retrain Model with Corrected Courtroom Data

Actor: ML Engineer **Preconditions:** Corrected and labeled courtroom sessions exist. Compute resources for training (e.g., a cluster or GPU server) are configured and accessible.

Main Flow:

1. User navigates to the “Training” or “Model Management” page.
2. System shows available labeled datasets (e.g., corrected sessions, curated corpora).
3. User selects one or more datasets to include in the new training run.
4. User configures training parameters such as model version name, number of epochs, and other hyperparameters.
5. User selects “Start Training”.
6. System packages the selected datasets and submits a training job to the compute backend.
7. System tracks training progress and displays status (e.g., “Queued,” “Running,” “Completed”).
8. When training completes, system registers the new model version and basic performance statistics.
9. User reviews results and may set the new model as the active model for future diarization sessions.

Alternate Flow: If the training job fails (e.g., due to resource limits or configuration errors), the system records the failure log and notifies the user. The user can adjust data selection or parameters and restart training. **Postconditions:** A new diarization model version is available in the system, tied to the training data and date, and may be deployed to improve future courtroom diarization accuracy.

Figure 3.5: Use Case 5: Retrain Model with Corrected Courtroom Data

3.4 Use Case Diagram(s)

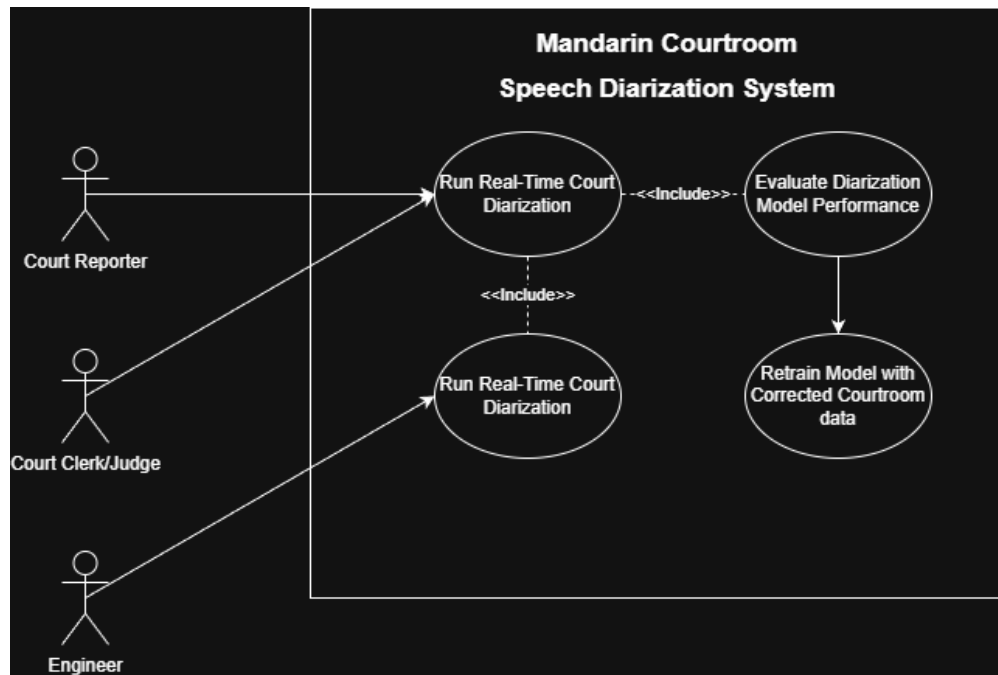


Figure 3.6: UML Use Case Diagram: ML Speech Diarization

3.5 Class Diagram

The Class Diagram represents the static structure of our senior design system and helps explain how each research component fits together before we run experiments. In our project, it maps the relationships between data collection, preprocessing, labeling, diarization models (baseline and fine-tuned), ASR, evaluation, and reporting modules. Writing it this way makes our architecture easier to justify in a research report because it shows that the project is not just “a model,” but an end-to-end pipeline with clearly separated responsibilities. It also helps us identify design risks early, like where overlap handling or data quality checks should be placed so errors do not propagate through the rest of the system.

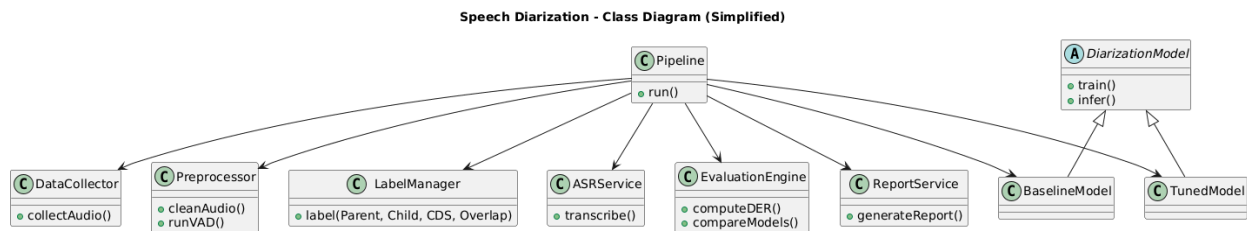


Figure 3.7: Class Diagram

3.6 Sequence Diagram

The Sequence Diagram explains the dynamic behavior of the system during an actual experiment run, which is important for reproducibility in research. It shows the order of actions from collecting raw parent-child audio, preprocessing it, running baseline and tuned inference, transcribing, and finally evaluating results. This diagram is useful because it makes the comparison process transparent, especially when we need to prove that both models were tested under the same conditions. In our report, this supports the credibility of our findings by showing exactly how outputs are produced and when decisions are made.

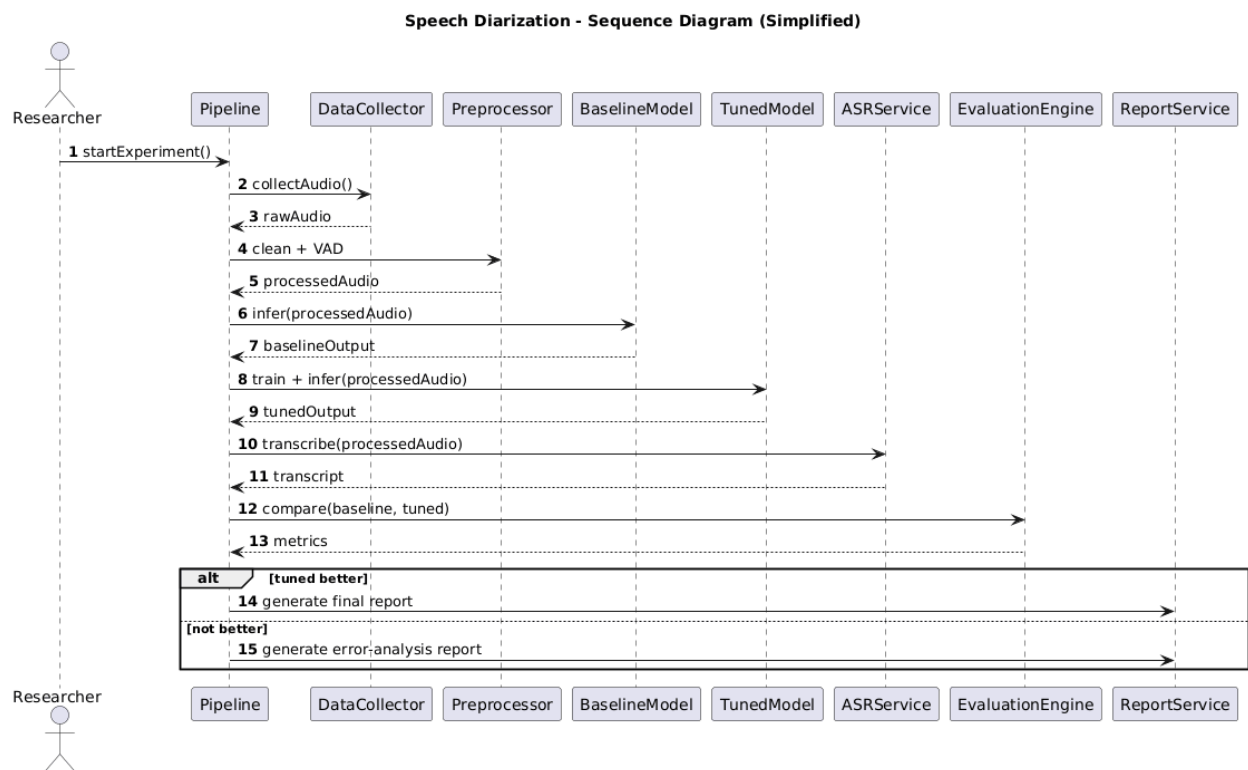


Figure 3.8: Sequence Diagram

3.7 Activity Diagram

The Activity Diagram presents the high-level research workflow and highlights the iterative nature of the project. Instead of showing object relationships or message timing, it captures the full process logic, including decision points like whether tuned performance improves over baseline. If results are weak, the workflow loops back to error analysis, data refinement, and retraining, which reflects how real research is actually done. For our senior design context, this diagram is especially helpful because it connects technical development with practical constraints such as noisy real-world audio, overlap-heavy segments, and compute scheduling limits.

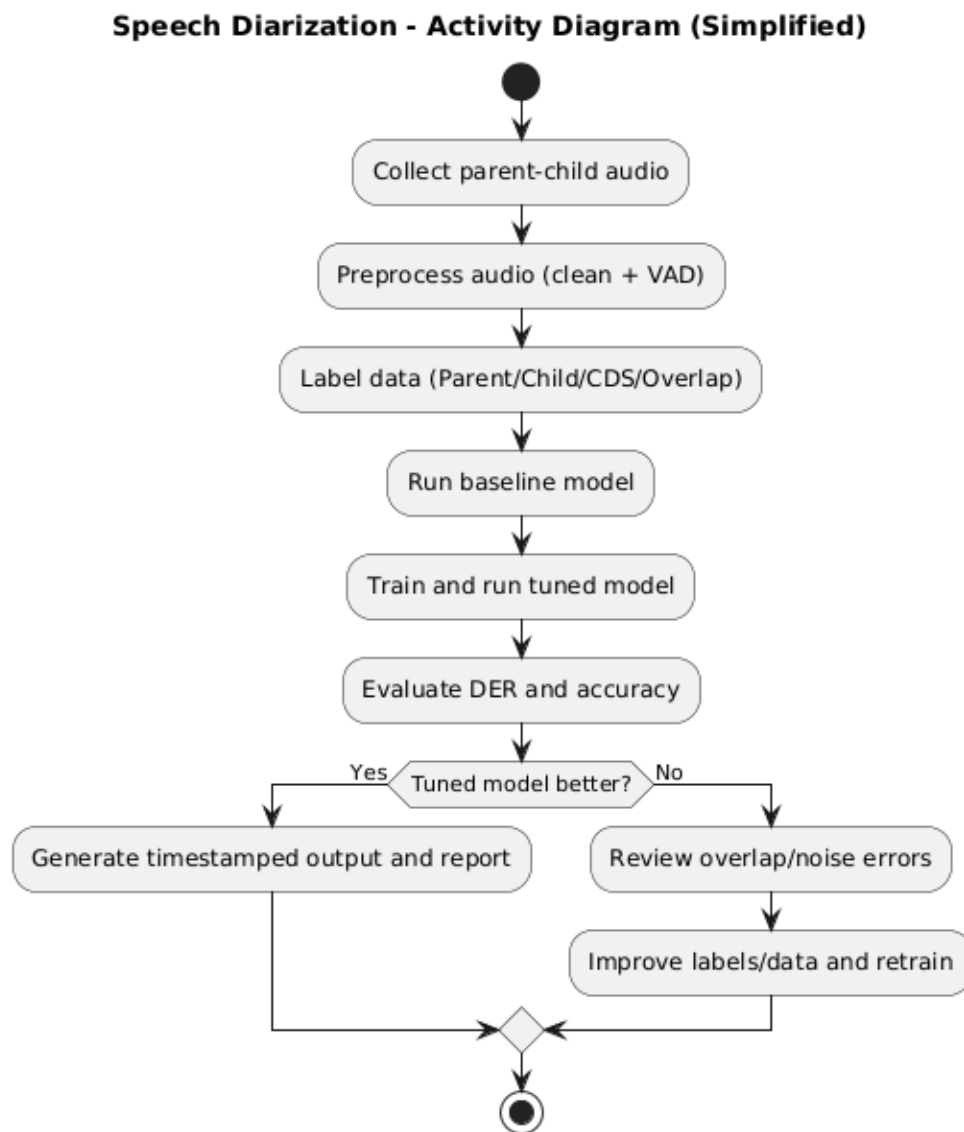


Figure 3.9: Activity Diagram

3.8 Functional requirements

System must perform automatic speaker diarization. The system must identify and differentiate individual speakers throughout an entire courtroom session, maintaining consistent speaker labels even when speakers change, interrupt, or speak simultaneously.

System must integrate Mandarin speech recognition. The system must process the diarized audio and produce a Mandarin-language transcript that accounts for tonal variation and reduces errors that commonly occur with homophones.

System must generate and store structured transcripts. The system must produce a timestamped, speaker-attributed transcript and store it in a retrievable format that can be reviewed, exported, and archived for legal use.

3.9 Non-functional requirements

System shall be accurate and reliable. The diarization and ASR components shall achieve a low diarization error rate and maintain performance despite courtroom conditions such as background noise, reverberation, and overlapping speech.

System shall be efficient and operate with low latency. The system shall process and output diarization and transcription results in near real-time, with minimal delay, to support live monitoring and rapid transcript verification.

System shall be usable and maintainable. The system shall provide a clean and intuitive interface suitable for legal staff, and its modular design shall allow future updates to machine learning models, datasets, or processing pipelines without requiring full system reconstruction.

3.10 Rationale

The system is designed as a research pipeline because the main purpose of the project is to study and improve diarization performance for child-adult speech. A more polished application with a complete user interface could make the project easier to demonstrate visually, but it would not directly address the most important technical problem. The main challenge is determining whether the system can separate adult speech, child speech, and overlapping speech

more accurately than a baseline model. For that reason, the design focuses on experimentation, evaluation, and reproducibility rather than production-level deployment.

One alternative design would have been to build a simple upload-based application where users submit audio and receive diarization results. While this approach would be useful in a finished product, it would limit the research value of the project. The system would still need a reliable pipeline behind it, and without strong evaluation, it would be difficult to prove whether the output was accurate. Since this project is focused on performance and improvement, the pipeline design is more appropriate than a user-facing application alone.

Another alternative would have been to rely entirely on an existing diarization model without building a project-specific workflow. This would reduce development time, but it would not fully address the unique challenges of child-adult audio. General diarization models are not always designed for children’s voices, child-directed speech, short turns, or frequent overlap. By creating a workflow that includes baseline testing, manual review, and repeated evaluation, the project can better identify where existing tools fail and what types of improvements are realistic.

The final design is a strong choice because it balances technical depth with the time limits of a senior design project. The modular structure allows the team to test individual stages while still evaluating the complete workflow. If an output is incorrect, the team can review whether the issue came from the original audio, preprocessing, diarization, or evaluation. This makes the system easier to debug and easier to explain in the thesis. It also creates a foundation for future work because later teams could improve one component, such as labeling, model training, or overlap detection, without replacing the entire system.

Chapter 4

Technologies - example middle chapter

4.1 System Components

The system is made up of several components that work together to support the full speech diarization workflow. The first component is the audio collection and input component. This part of the system gathers child-adult audio clips and organizes them for experimentation. In this project, audio may come from public video sources or other approved sources that contain child-adult interaction. Once the audio is collected, files must be stored in a consistent structure so that they can be used for baseline testing, improved testing, and final evaluation.

The second component is the preprocessing component. This component prepares the raw audio for diarization by checking file formats, converting files when needed, trimming or segmenting longer clips, and maintaining timestamp consistency. Preprocessing is especially important for this project because child-adult audio can be noisy and unpredictable. Background noise, interruptions, and overlapping voices can all affect model performance. A clear preprocessing stage helps reduce avoidable errors and makes the later evaluation more reliable.

The third component is the diarization model component. This part of the system identifies speaker segments and produces timestamped labels that show who spoke when. Existing diarization tools can be used as baseline systems, and their outputs can be compared against improved versions of the pipeline. In this project, the most important challenge is separating adult speech, child speech, child-directed speech, and overlapping speech. The diarization component is therefore the core technical part of the system.

The fourth component is the automatic speech recognition component, if it is used in a specific experiment. Although the project focuses mainly on diarization rather than transcription, ASR can provide helpful context when reviewing audio segments. A transcript can make it easier to understand where speaker changes occur and whether the diarization output makes sense. However, ASR is treated as a supporting component rather than the main goal of the project.

The fifth component is the evaluation component. This component compares diarization output against manually reviewed labels or selected ground-truth segments. The main evaluation metric is Diarization Error Rate, but the project also considers overlap handling and label quality. This component is necessary because the project needs evidence to show whether the system improves over the baseline. Without evaluation, the team could only describe the outputs subjectively.

The sixth component is the compute and runtime environment. Diarization and model testing may require significant computing resources, so the project uses available systems such as WAVE/HPC when needed. This component includes job scripts, runtime logs, dependencies, GPU access, and scheduling through the cluster environment. Because these resources are not always fully controlled by the team, the system must be designed so that experiments can be repeated and verified even when compute access is limited.

The final component is the output and reporting component. This part of the system stores diarization results, evaluation summaries, transcript files when available, and logs from each experiment. These outputs allow the team to compare results across different runs and document the final findings in the thesis. Since this project is a research prototype, clear reporting is essential. The value of the system comes not only from producing output, but from producing results that can be reviewed, explained, and used to guide future work.

Chapter 5

System Evaluation - example middle chapter

What we are evaluating. The system will be evaluated on its ability to:

- Collect and process parent–child audio data (e.g., extracted from public videos).
- Produce diarization outputs with clear timestamps and speaker segments.
- Identify project-specific cases such as child speech, child*-directed speech, and overlapping speech.

Evaluation approach. We will follow a three-phase evaluation process:

1. **Baseline testing:** Run a baseline diarization pipeline (e.g., Pyannote + Whisper) on the same evaluation clips to establish baseline performance.
2. **Data validation:** Verify that audio downloads are usable, timestamps align correctly through preprocessing, and labels match the spoken content on a manually checked subset.
3. **Final comparison:** Run the improved (fine-tuned and/or modified) system on the same evaluation set and compare results directly to the baseline.

Metrics and success criteria. Our primary metric will be **Diarization Error Rate (DER)** on a held-out evaluation set. We will also include overlap-focused scoring because overlap is a major failure point in parent–child conversations. Success is defined as:

- Lower DER than the baseline on the same evaluation set.
- Improved handling of overlapping segments (fewer missed/incorrect overlap cases).

- Better label quality on a manually labeled subset for key categories (child speech, child-directed speech, overlap).

Test dataset strategy. To ensure realistic evaluation, we will include clips that represent:

- Clean vs. noisy audio,
- High-pitched/variable child vocalizations,
- Frequent interruptions and overlapping speech.

This helps confirm that improvements generalize beyond ideal audio conditions.

Ethics and data handling. Because the data involves children, we will limit access to approved team members, avoid collecting personal identifiers, and store data securely. The evaluation will report results at an aggregate level and focus on model performance rather than individual speakers.

5.1 Internal Testing

Internal testing validates correctness, stability, and reproducibility before external feedback. It includes unit tests, integration tests, and repeated evaluation runs.

Unit testing (component-level). We will test each module independently:

- **Scraping and audio extraction:** confirm files download correctly, audio is extracted in the expected format, and failures are logged.
- **Preprocessing and segmentation:** verify trimming/chunking consistency and timestamp alignment.
- **Labeling pipeline:** spot-check a subset of labeled segments against the audio to confirm label correctness.
- **Inference output:** confirm the diarization output format is correct (timestamped segments and speaker labels) and consistent across repeated runs.
- **Compute environment checks:** confirm training/inference runs correctly in the target environment (e.g., WAVE/HPC), including dependencies and GPU usage if applicable.

Integration testing (end-to-end). After unit tests, we will run full pipeline tests:

- End-to-end: collect → preprocess → label subset → baseline run → improved run → metrics report.
- Smoke tests on a small set (e.g., 5–10 clips) after major code changes to ensure nothing breaks.

- Reproducibility checks using fixed evaluation clip lists and saved configurations.

Internal acceptance criteria. Internal testing is successful when the full pipeline runs reliably, outputs are formatted correctly, and the improved system shows consistent metric gains over baseline on internal evaluation runs.

5.2 External Testing

External testing evaluates whether the system results and evaluation are credible to people outside the development team, and it provides feedback for iteration.

External reviewers. External reviewers may include our advisor/instructor, classmates, or other technical reviewers familiar with speech/audio ML. They will not modify the system, but they will review outputs and evaluation logic.

External testing methods.

- **Methodology review:** provide a short evaluation packet describing dataset selection, baseline definition, metrics, and success criteria; ask whether the evaluation is fair and repeatable.
- **Qualitative output audit:** share a small set of clips with predicted diarization logs (and ground-truth labels where appropriate) and ask reviewers to identify obvious failure modes such as misassigned child speech or missed overlap.
- **Output usability review:** check whether the timestamped output is clear and usable for downstream analysis (e.g., reading, searching, or summarizing who spoke when).
- **Ethics review:** confirm that privacy and responsible data handling are clearly documented and reasonable for the project context.

Incorporating feedback. We will log external feedback as issues (problem, example evidence, proposed fix) and prioritize improvements that:

- Reduce overlap and speaker confusion errors,
- Improve child speech vs. adult speech separation,
- Increase clarity and consistency of output formatting.

External success criteria. External testing is successful when reviewers agree the evaluation plan is understandable and defensible, outputs are interpretable, and the final system demonstrates measurable improvement over baseline results.

5.3 Unit Testing

```
• (.venv) [kkang@login02 SeniorDesign]$ cd /WAVE/users2/unix/kkang/SeniorDesign
./run_tests.sh
test_is_audio_file_accepts_audio_and_rejects_generated_inputs (test_project_utilities.NumberAudioInFilesTests) ... ok
test_list_audio_files_returns_sorted_audio_only (test_project_utilities.NumberAudioInFilesTests) ... ok
test_main_renumbers_unprefixed_audio_files (test_project_utilities.NumberAudioInFilesTests) ... ok
test_next_available_number_returns_first_gap (test_project_utilities.NumberAudioInFilesTests) ... ok
test_collect_used_prefix_numbers_ignores_invalid_candidates (test_project_utilities.YoutubeToAudioBatchTests) ... ok
test_ensure_numbered_audio_path_renames_unprefixed_file (test_project_utilities.YoutubeToAudioBatchTests) ... ok
test_extract_video_id_handles_standard_and_short_urls (test_project_utilities.YoutubeToAudioBatchTests) ... ok
test_prune_failed_urls_from_list_removes_only_failed_entries (test_project_utilities.YoutubeToAudioBatchTests) ... ok
test_resolve_audio_path_in_dir_accepts_only_files_inside_output_dir (test_project_utilities.YoutubeToAudioBatchTests) ... ok

-----
Ran 9 tests in 0.002s

OK
```

Unit testing is only partially necessary for our speech diarization project. It is useful for checking deterministic parts of the pipeline, such as downloading YouTube audio, pre-processing files, parsing model output, and handling invalid inputs. However, unit tests alone cannot show whether the diarization system is actually accurate for parent-directed and child speech, since that depends on model quality, training data, and real audio conditions. In our project, the main problem is not that the code fails to run, but that the diarization results are poor. Because of that, evaluation on labeled audio samples and end-to-end testing are more important than relying only on traditional unit tests.

5.4 Integration Test(s)

Integration testing is needed for this project because the system is a pipeline made of multiple cooperating modules, not a single isolated program. GeeksforGeeks defines integration testing as testing combined units after unit testing so that interface defects can be found. That applies directly to this project because success depends on the handoff between scripts and runtime environments: YouTube links must become audio files, audio files must be correctly numbered, the numbering must stay consistent with the URL index, and the diarization job must read those files and produce transcript outputs. A unit test can confirm that one helper function works, but it cannot prove that the outputs of one module are valid inputs for the next module.

For this reason, the project should use an incremental integration strategy. The most practical approach is to automate the local file-based part of the pipeline and describe the cluster-dependent part as a manual integration procedure. A fully automated end-to-end test for every backend is not necessary for this assignment because the final stage depends on external factors such as SLURM scheduling, GPU availability, Hugging Face model access, Python environment installation, and network access for downloading packages or media. Those dependencies make a full-stack automated test expensive and brittle. However, the interfaces between the project's own modules still need to be verified.

The automated integration test added for this project is in `'tests/test_integration_pipeline.py'`. This

test combines 'number_audio_in_files.py' and 'youtube_to_audio_batch.py' in one workflow. The test starts with one existing unnumbered audio file in 'audio_in/' and one YouTube URL in a temporary links file. First, it runs the numbering script and verifies that the existing file becomes '001...'. Next, it runs the YouTube batch script with the actual file-system behavior but a mocked download function, so the test stays fast and deterministic. The test then verifies that the newly downloaded audio file is renamed to '002...', the URL-to-audio index file is updated, the resolved-audio list is written, and the recorded metadata matches the downloaded item. This confirms that the boundary between numbering, downloading, renaming, and indexing works correctly.

The second integration test for the project should be a manual end-to-end procedure. First, run 'youtube_to_audio_only.sbatch' and verify that the job logs are created, the resolved audio paths all point inside 'audio_in/', and the downloaded files are numbered consistently. Second, run 'submit_bulk_diarization.sh --backend nemo' or '--backend pyannote' and confirm that each numbered audio file produces both a '.txt' transcript and a '.srt' subtitle file in the job output directory. Also verify that 'runtime_summary.tsv' lists every processed audio file and that no output path escapes the expected folders. This manual test is appropriate because the assignment explicitly allows manual integration testing and because the diarization stage depends on cluster resources and third-party models.

In summary, integration testing is needed for this project because the main risk is not only whether each script works by itself, but whether the scripts work together as one system. What is not necessary is a fully automated cluster-wide end-to-end test for every backend and model combination. For this assignment, one automated integration test for the project-owned pipeline logic plus one documented manual full-system procedure is the correct balance.

Chapter 6

Implementation Plan - example middle chapter

6.1 Timeline

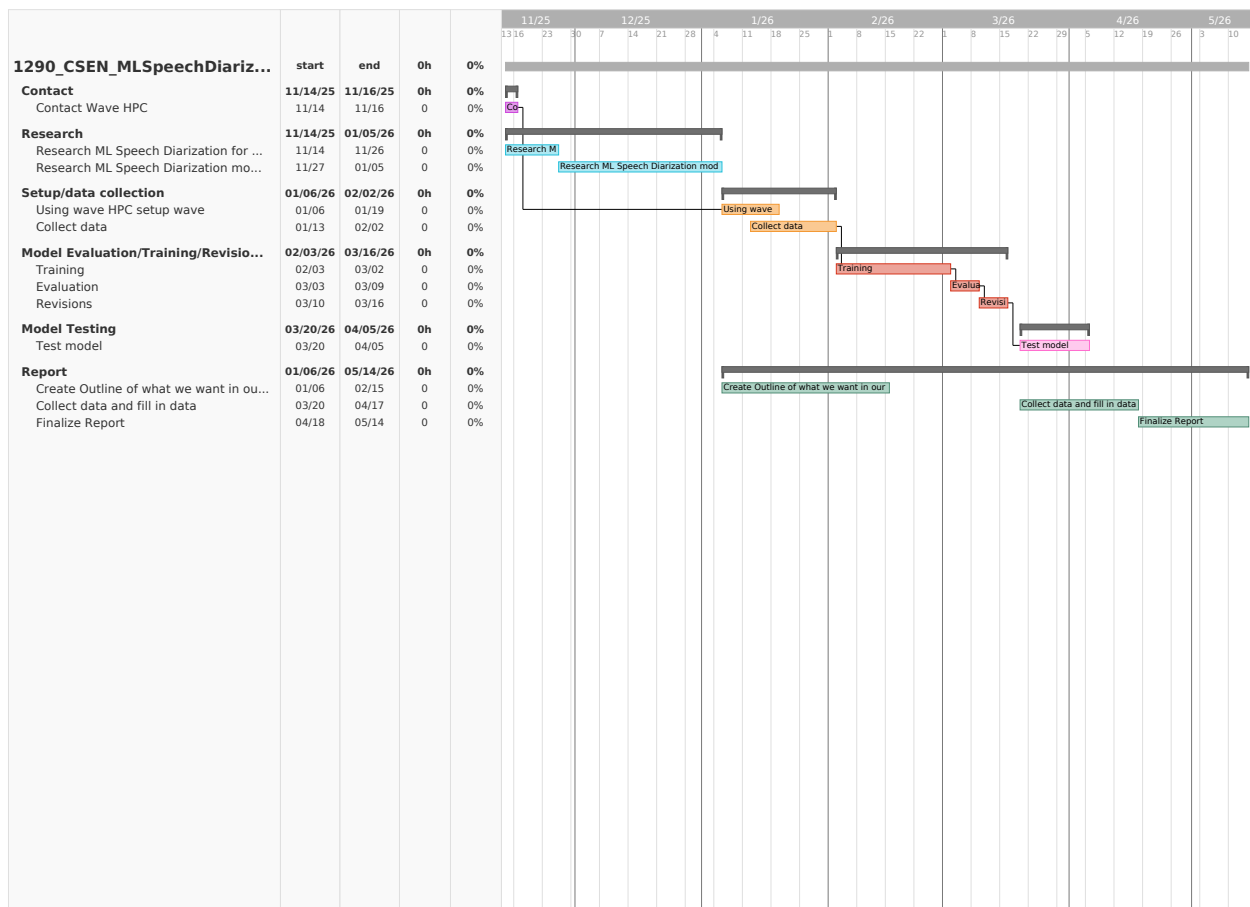


Figure 6.1: ML Speech Diarization Timeline Diagram

6.2 Project Risks

- **Limited time frame**

- **Risks:** Major risk is not completing the project in time. This is due to complications during our previous quarter.
- **Impact:** The quality of the project may be impacted and also overall grade for senior design.
- **Mitigation:** Make sure there is work being completed every week.

- **Model limitations**

- **Risks:** If the technology being used is limited our results for Child Directed Diarization may not be optimal. This means that the AI model is limited to the actual software rather than the training methods.
- **Impact:** Project may come out as a failure but also gives us insight of how future developers can improve the product.
- **Mitigation:** Research into the newest technology that can be trained and make sure that training techniques and software is the most current.

- **HPC Wave Queue**

- **Risks:** Queue might be filled when running programs (Depending on other projects that use HPC wave) which could result in delays in certain trainings.
- **Impact:** Slight impact on speed depending on how long delays are but could also yield training for long periods at a time.
- **Mitigation:** Use HPC wave as soon as possible to reduce amount of delay that is out of our control.

- **Privacy**

- **Risks:** There is a big concern about privacy, usage of that data, and protection of the data. This is because the data we are using is public and the demographic includes children.
- **Impact:** Product could garner a lot of backlash because of how the data was obtained.
- **Mitigation:** Primarily use open source data and self generated data that can protect individuals. Also limiting those who have access to data is very important to mitigating risk of privacy leak.

- **Algorithmic Bias**

- **Risks:** Data might be bias based on the material given. This means that the diarization product may be bias against accents, languages, or even regional dialects.
- **Impact:** Diarization product may be limited to specific situations or only work during certain conditions.
- **Mitigation:** Give a wider ranger of data that covers all components may help mitigate algorithmic bias.

Chapter 7

Constraints and Standards

7.1 Constraints

Constraint 1: Data availability and labeling quality

One major constraint in this project is the availability of suitable child-adult speech data and the amount of work required to label it well. Our evaluation section already shows that the project depends on clips that include clean and noisy audio, high-pitched and variable child vocalizations, and frequent interruptions and overlap. Those are exactly the cases that make diarization difficult, but they are also the cases that are hardest to label consistently. Because of this, our team cannot assume that more data automatically means better data. A smaller, carefully checked dataset may be more useful than a larger dataset with weak labels. This constraint affects model training, evaluation fairness, and the strength of any conclusions we draw from the final results.

Constraint 2: Privacy and legal handling of child audio

Privacy is another major constraint because this project involves recordings that include children. In the current draft, we already note the need to limit access to approved team members, avoid personal identifiers, and store data securely. That makes privacy a design constraint, not just an ethical discussion topic. In addition, child-focused data collection can trigger stricter legal expectations. The FTC states that COPPA imposes requirements on operators of websites or online services directed to children under 13, and on operators with actual knowledge that they are collecting personal information online from a child under 13 [1]. Our prototype is a senior design research system rather than a public commercial product, so we are not claiming direct COPPA compliance here. However, the existence of COPPA still matters because it shows that any future deployment or direct online collection involving children would face additional legal limits on what data could be collected, retained, or shared [1].

Constraint 3: Compute and infrastructure limits

Our project is also constrained by compute availability and infrastructure reliability. The thesis already identifies the WAVE/HPC queue as a project risk, and the testing chapter explains that full end-to-end automation is difficult because the system depends on SLURM scheduling, GPU availability, model access, Python environment setup, and other external runtime conditions. This means our technical progress is partly limited by infrastructure that we do not fully control. Even if our code is correct, delays in the queue or missing dependencies can slow training, testing, and revision. Because of that, our design has to balance ambition with practicality and avoid assuming unlimited compute or perfectly available backend resources.

Constraint 4: Senior design time and scope limits

A fourth constraint is the limited time frame of the senior design project itself. The implementation timeline spreads work across research, setup, data collection, training, evaluation, testing, and report finalization over a single academic schedule. The risks section also directly states that limited time is a major concern. In practice, that means our project cannot fully explore every model architecture, every dataset option, or every possible improvement cycle. We have to make decisions about scope, such as focusing on the most important failure cases like child speech, child-directed speech, and overlap, instead of trying to solve every speech diarization problem at once. This time constraint also affects how much manual validation we can do and how polished the final prototype can realistically become.

Constraint 5: Model generalization and reliability

A final important constraint is that speech diarization for parent-child interactions is technically difficult even when the pipeline is working correctly. The evaluation section already explains that overlap is a major failure point and that realistic testing must include noisy audio, variable child vocalizations, and frequent interruptions. The risks section also notes model limitations and algorithmic bias. Together, these issues constrain how confidently we can generalize results beyond the evaluation set. Even a model that improves DER on our selected clips may still struggle on new accents, recording conditions, or speech patterns that were underrepresented in training. Because of this, the project has to be framed as an iterative research prototype rather than a finished production system.

7.2 Standards

Standard 1: ISO/IEC/IEEE 29148:2018, Requirements Engineering

ISO/IEC/IEEE 29148:2018 is a strong fit for this project because it specifies required processes and information items for requirements engineering across the system lifecycle [2]. That matters here because our thesis already separates functional and non-functional requirements, and those requirements need to be specific enough to support later testing and evaluation. In our project, this standard helps justify writing requirements that are clear, verifiable, and traceable. For example, goals such as producing timestamped speaker segments, maintaining reliability under overlap and noise, and keeping the system maintainable are stronger when they are treated as formal engineering requirements instead of informal project wishes [2].

Standard 2: ISO/IEC/IEEE 29119-1:2022, Software Testing — General Concepts

ISO/IEC/IEEE 29119-1:2022 is relevant because it specifies general concepts in software testing for the 29119 series [3]. Our thesis already includes unit testing, integration testing, smoke testing, reproducibility checks, and manual end-to-end testing. Using this standard as a reference strengthens Chapter 5 and Chapter 7 by showing that our testing approach is not ad hoc. Instead, it follows an established software testing framework that fits a multi-stage pipeline. This is especially important in our project because correctness is not only about whether a script runs, but also whether the modules connect reliably and whether evaluation results can be reproduced [3].

Standard 3: ISO/IEC 27001:2022, Information Security Management Systems

ISO/IEC 27001:2022 is relevant because it defines requirements for an information security management system [4]. Even though our project is a student prototype, we are still handling potentially sensitive audio involving children. That means security decisions such as limiting access, protecting stored files, and managing risk around data leakage are not optional design details. Referencing ISO/IEC 27001 helps support the idea that secure handling of data should be systematic rather than informal. In the context of our project, this standard is most relevant to how we store audio, control access to datasets, and reduce the chances of privacy leaks during experimentation [4].

Standard 4: ISO/IEC 29100:2024, Privacy Framework

ISO/IEC 29100:2024 is also highly relevant because it provides a privacy framework, including common privacy

terminology, roles in processing personally identifiable information, and privacy safeguarding considerations [5]. That matches the needs of this project well because our work involves child-adult recordings where privacy cannot be treated casually. Even if we avoid collecting obvious identifiers, voice data can still be sensitive, and team members need a clear framework for what data is collected, who can access it, and how it should be protected. This standard supports our decision to minimize identifiers, define data-handling responsibilities more clearly, and treat privacy as a core systems issue rather than only an ethics paragraph [5].

Standard 5: ISO 8601-1:2019, Date and Time Representations for Information Interchange

ISO 8601-1:2019 is relevant because it specifies standardized representations of dates and times for information interchange, including UTC-based time shifts [6]. That matters for speech diarization because our outputs depend on precise timing. Segment boundaries, experiment logs, evaluation results, and exported metadata all become easier to interpret and compare when timestamps are stored in a consistent format. In our project, using a standard timestamp convention would reduce ambiguity when matching speaker segments to transcripts, comparing runs across systems, and storing experiment history for later review [6].

Chapter 8

Societal Issues

8.1 Ethical

Ethical issues are a major part of our project because we are working with speech data that involves children, which makes privacy and responsible data handling especially important. Even though our system is focused on diarization, meaning identifying who spoke when, the recordings still capture real conversations and vocal characteristics that should be treated carefully. For that reason, our project should avoid collecting personal identifiers, limit access to the data, and report results at the aggregate level instead of focusing on specific individuals. Another ethical concern is model bias. If our training data does not include enough variation in children’s voices, adult voices, accents, background noise, and overlapping speech, then the system may work better for some cases than others. That would make the results less fair and less trustworthy. As engineers, we have a responsibility not only to improve performance, but also to be honest about limitations and avoid presenting the model as more accurate than it really is.

8.2 Social

Our project also has an important social impact because it deals with how communication between children and adults is analyzed and understood. A more accurate diarization system could help researchers and educators study interaction patterns more efficiently, especially in situations where there is a lot of interruption, overlap, or unclear turn-taking. That could make speech-based analysis more accessible and less time-consuming. At the same time, there are social concerns that come with using this type of system. If people start relying too heavily on automated outputs, they may treat the model’s labels as objective truth even when the system still makes mistakes. This is especially important in parent-child speech, where context matters and errors can change the meaning of an interaction. Because of that, the social value of our project depends on using the technology as a support tool for analysis rather than as a replacement for human judgment.

8.3 Economic

There are also economic issues connected to our project. One of the main motivations for speech diarization is that manually labeling long recordings takes a large amount of time and effort. If our system can separate child and adult speech more accurately, it can reduce the amount of manual correction needed and make future research workflows more efficient. In that sense, the project could lower labor costs and save time. However, the system also comes with costs of its own. Training and evaluating machine learning models requires compute resources, storage, repeated testing, and time spent validating labels. Our thesis also points out practical limitations such as HPC queue delays and model limitations, which show that better performance is not free. From an economic perspective, the project is only valuable if the improvement in diarization quality is large enough to justify the costs of collecting data, running experiments, and revising the model over time.

8.4 Health and Safety

Even though our project is software-based and not a physical product, health and safety issues still apply. In this case, the biggest concerns are related to privacy, misuse, and incorrect interpretation of results. Since the data involves children, mishandling recordings could create risks for confidentiality and public trust. That means data storage and access control are part of the safety responsibility of the project. There is also a safety issue in how the output is used. If the model performs poorly on noisy or overlap-heavy recordings, users might still assume the output is accurate and use it to make conclusions that are not fully supported by the data. In research settings related to child development or communication analysis, that could be harmful. Because of this, our system should be treated as an assistive tool that helps with analysis, while still keeping humans involved in reviewing uncertain or difficult segments.

8.5 Usability

Usability is one of the most important societal issues for our project because even a strong model is not very helpful if the output is difficult to understand or work with. Our system is meant to produce timestamped speaker segments clearly enough that users can review, compare, and analyze the results. Since child-adult conversations often include interruptions, overlap, and less predictable speech patterns, the interface and output format need to make those difficult cases easy to inspect. A usable system should let the user quickly understand where speaker changes happen, where the model may be uncertain, and which parts may need manual correction. In other words, usability is not just about making the interface look clean. It is about making sure the system communicates results in a way that is practical, transparent, and easy to interpret. If the output is confusing or hides uncertainty, then even good model performance

will not be enough to make the system trustworthy in real use.

8.6 Compassion

Compassion is relevant to our project because it reminds us that the audio we process represents real people, not just data points. Since our work focuses on speech interactions between children and adults, we should be careful not to treat difficult speech patterns as just a technical inconvenience. Children’s speech can be highly variable, less stable, and harder for existing systems to classify, but that should motivate better system design rather than unfair judgment about the speakers themselves. A compassionate engineering approach means building technology that respects the dignity of the people represented in the data and recognizes that there are human consequences behind every design decision. In our project, compassion also means asking why we are building this system in the first place. If it can help reduce repetitive manual work, improve the study of communication, and support more accurate analysis without invading privacy, then it can provide real value. Keeping compassion in mind helps ensure that the project stays centered on people rather than only on performance metrics.

Chapter 9

Conclusion

State what you learned from your work. In this section:

- Summarize what you did. This can be viewed as the evidence.
- State what you learned (the actual conclusions that you are drawing), and relate them to the project objectives.
- List the advantages and disadvantages of your work. In what ways is your solution deficient or lacking?
You are not divulging a weakness in your work when you state problems that still remain.
- State directions for future work and list any open problems.
- **Do not use bullet points!**

Don't forget to delete this explanatory red text/block introduction, and replace it with your text in black.

Chapter 10

Acknowledgments

Acknowledge the contributions of the sponsor, university staff, other students, faculty, and other persons who were of assistance.

This section is OPTIONAL.

Don't forget to delete this explanatory red text/block introduction, and replace it with your text in black.

Chapter 11

References

- [1] Federal Trade Commission, “Children’s Online Privacy Protection Rule (“COPPA”),” Federal Trade Commission, 2025.
- [2] International Organization for Standardization, “ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering,” ISO, 2018.
- [3] International Organization for Standardization, “ISO/IEC/IEEE 29119-1:2022, Software and systems engineering — Software testing — Part 1: General concepts,” ISO, 2022.
- [4] International Organization for Standardization, “ISO/IEC 27001:2022, Information security, cybersecurity and privacy protection — Information security management systems — Requirements,” ISO, 2022.
- [5] International Organization for Standardization, “ISO/IEC 29100:2024, Information technology — Security techniques — Privacy framework,” ISO, 2024.
- [6] International Organization for Standardization, “ISO 8601-1:2019, Date and time — Representations for information interchange — Part 1: Basic rules,” ISO, 2019.

Chapter 12

Appendices

Include:

- Important source code listings (listing 50 pages of source code is not really helpful), as a rule of thumb make it around 5 to 10 pages of source code listings, not more.
- Logic diagrams
- Parts lists
- Parts layout
- Data tables
- Background calculations
- And other information needed for completeness, but would bog down the discussion in the body of the report.

Don't forget to delete this explanatory red text/block introduction, and replace it with your text in black.